SAMRIT: MASTER PRD INTRODUCTION

Name from Samaritan (literal, and also it evokes the Good Samaritan Food Donation Act) + Amrit (Sanskrit for the "nectar of immortality" or life-giving sustenance)

1. General Problem Statement

Every year, millions of tons of edible surplus food from corporate events, conferences, and catering operations are thrown into landfills while local communities struggle with food insecurity. The primary barrier to food donation is not a lack of empathy, but **logistical friction and liability fears**.

Existing food rescue applications operate essentially as glorified bulletin boards—they connect a donor to a soup kitchen but push the entire burden of inventorying, food safety tracking, and logistics onto understaffed NGOs. When heavy industrial catering is involved, event organizers find it faster and legally safer to trash the food than to manually log allergens, verify temperature requirements, and coordinate driver dispatch. There is a critical need for an intelligent system that automates the friction of ingestion, guarantees a verifiable chain of custody for food safety, and algorithmically routes perishable goods based on hardware constraints (e.g., matching a need for refrigeration with a facility that actually has refrigerators).

2. Product Description

CivicBite is an AI-assisted food recovery and logistics platform designed to eliminate the operational friction of food donation. By leveraging multimodal Large Language Models (LLMs) and a strict geospatial tracking architecture, the system creates a seamless, end-to-end pipeline from the event floor to the community meal.

The platform orchestrates four interconnected user "Faces":

- **Face 1 (Consumers):** A mobile-first interface providing dignified discovery and dietary-safe matching, allowing users to filter available centers by FDA-recognized allergens. It includes transit navigation and a mechanism for leaving feedback.
- **Face 2 (Distribution Centers):** A facility-profiling portal where soup kitchens and shelters register their operating hours, seating, and available appliances (such as refrigerators or warmers). Center staff use this interface to scan incoming QR codes and complete the final intake verification.
- **Face 3 (Event Organizers / Donors):** A frictionless donation app where organizers snap photos of catering menus, allowing an LLM to automatically extract dishes and allergens for human verification. The LLM also utilizes predictive logistics to infer transport requirements directly from the food type (e.g., identifying that ice cream requires deep-freeze transport).

- **Face 4 (Central NGO / Logistics):** An AI-driven dispatch engine. It features a driver workflow where volunteers scan event QR codes and assign unique, pre-printed QR tracking stickers to physical food containers. The backend executes algorithmic routing to match food lifespan constraints (hot vs. cold) with the registered appliances of nearby distribution centers.

Core System Mandates (The Chain of Custody)

To ensure absolute data integrity and food safety, the architecture strictly enforces the following global rules across all modules:

1. **Visual Quality Management:** The system requires photographic proof of the food at three distinct lifecycle stages: at the event by the donor, at pickup by the driver, and at drop-off by the distribution center.
2. **Geospatial Verification:** Every time a QR code is scanned within the system, the device's GPS Lat/Long is automatically logged to prevent fraud and create a fully auditable chain of custody.
3. **Internal Rating Engine:** All quality ratings and feedback submitted by consumers or centers are parsed by an LLM to manage a priority queue and flag bad actors; however, these ratings are visible strictly to the Central NGO backend and are never displayed publicly.

(Team: Paste this section at the very top of your AI prompts to establish the system's global context, then append your specific Module instructions from Parts 2 and 3 below it.)

1. Executive Summary

Samrit is an intelligent supply chain and logistics platform that rescues surplus food from corporate events and redistributes it to community soup kitchens/shelters. It replaces manual coordination with an AI-driven pipeline that handles inventory OCR, constraint-based algorithmic matching (cold/hot storage), dynamic driver routing, and a strict 3-stage photographic chain of custody using QR codes.

2. System Architecture & Tech Stack

- **Frontend (Faces 1, 2, 3, 4):** Next.js (React), Tailwind CSS, TypeScript. Built as Progressive Web Apps (PWAs) for mobile camera access without App Store deployment.
- **Backend Engine:** Node.js, Express.js, TypeScript.
- **Database:** MongoDB via Mongoose.
- **Validation Contract:** Zod (Used across frontend forms, backend APIs, and LLM output parsing).
- **AI Architecture (Hybrid):**
    - *Local AI Gateway (Qwen 9B):* Acts as a router, prompt compressor, and JSON validator.
    - *Cloud AI (Gemini 3.8 Flash):* Handles Vision (Menu OCR, Food photos) and complex text synthesis.
    - *API Load Balancing:* Backend rotates 4 separate API keys to maximize Free Tier Limits (TPM/RPM).

3. Universal Integration Standards

All AI coding assistants must be instructed to follow these strict rules to prevent integration conflicts.

- **Communication Protocol:** RESTful API. All payloads must be strictly `application/json`.
- **Standardized API Response Wrapper:** Every backend route MUST return data in this exact format:
- TypeScript

```
{
  success: boolean;
  data?: any; // The requested payload
```

```
  error?: string; // Human readable error message
  code: number; // HTTP Status Code
}
```

- **State Management:** React Context API for global states (Auth, User Role). Local component state (`useState`) for forms. No Redux.
- **Chain of Custody Rule:** Any time a QR code is scanned or a photo is uploaded, the frontend MUST append the device's GPS coordinates `{ lat: number, lng: number }` to the API payload.

4. Core Data Models (The Single Source of Truth)

Member 1 will implement these in Mongoose. All other members will feed these schemas into their AI assistants so the frontend knows exactly what JSON to send/expect.

1. `FoodBatch` (The core entity moving through the system)

```
TypeScript
{
  batchId: string, // UUID
  donorEventId: string, // Links to the event
  qrStickerId: string | null, // The physical sticker linked by the driver at pickup
  status: "pending" | "matched" | "in_transit" | "delivered" | "rejected",
  handlingRequirement: "hot" | "refrigerated" | "shelf_stable",
  items: Array<{
    name: string,
    estimatedServings: number,
    allergens: string[] // FDA recognized (e.g., "Dairy", "Nuts")
  }>,
  chainOfCustody: {
    stage1_DonorPic: { url: string, timestamp: Date, lat: number, lng: number },
    stage2_PickupPic: { url: string, timestamp: Date, lat: number, lng: number } | null,
    stage3_DropoffPic: { url: string, timestamp: Date, lat: number, lng: number } | null
  }
}
```

2. `CenterProfile` (The Soup Kitchen / Shelter)

TypeScript

```
{
  centerId: string,
  name: string,
  location: { lat: number, lng: number, address: string },
  capabilities: {
    refrigeration: boolean,
    hotHolding: boolean,
    maxMealCapacity: number
  },
  operatingHours: { open: string, close: string }, // "HH:MM" format
  locationQrCode: string // The static QR code pasted on their front door
}
```

3. `DispatchRoute` (The Driver's Assignment)

TypeScript

```
{
  routeId: string,
  driverId: string,
  status: "active" | "completed",
  assignedBatches: string[], // Array of FoodBatch IDs
  waypoints: Array<{
    type: "pickup" | "dropoff",
    location: { lat: number, lng: number },
    batchId: string,
    status: "pending" | "done"
  }>
}
```

4. `FeedbackReview` (The Internal NGO Rating System)

```
TypeScript
{
  reviewId: string,
  targetType: "donor" | "center", // Who is being reviewed
  targetId: string,
  linkedBatchId: string | null, // If the review is about specific food
  sentiment: "positive" | "negative",
  rawText: string, // What the user typed
  aiParsedFlags: string[] // e.g., ["spoiled_food", "late_pickup", "unsafe_packaging"]
}
```

Module 1: Core Platform & Data (Team Member 1)

Role: Backend & Database Architect

Tech Stack: Node.js, Express, Mongoose, Zod.

Scope & Boundaries:

You are the gatekeeper of the database and the AI Gateway. You build the foundational Express server, define the Mongoose schemas (from Part 1), and expose the basic CRUD REST APIs. You also build the API load balancer that rotates the 4 Gemini API keys. You do **not** write algorithmic matching logic, and you do **not** build any UI.

Core API Endpoints (CRUD & AI Gateway):

All responses must strictly adhere to the { success, data, error, code } wrapper.

- POST /api/batches
  - **Input:** JSON matching the FoodBatch Zod schema (minus batchId and status, which the backend assigns).
  - **Output:** Returns the created FoodBatch object.
- PUT /api/batches/:batchId/photo
  - **Input:** multipart/form-data containing the image file, stage (1, 2, or 3), lat (number), and lng (number).
  - **Output:** Updated FoodBatch object with the new chainOfCustody URL and timestamp.

- POST /api/llm/ocr (The AI Vision Gateway)
    - **Input:** multipart/form-data containing a photo of a catering menu.
    - **Logic:** Server selects an available Gemini 3.8 Flash API key, sends the image, and requests a strict JSON output of dishes and allergens.
    - **Output:** { "items": [{ "name": "Pasta", "allergens": ["Gluten"] }] }

LLM Integration Points:

- **The 4-Key Rotator:** Create a middleware utility array: const keys = [process.env.KEY1, process.env.KEY2, process.env.KEY3, process.env.KEY4]. Store an index in memory and increment it modulo 4 on every request to distribute the Tokens Per Minute (TPM) limit evenly.

AI Prompting Instructions for Member 1:

"Act as a strict Node.js/TypeScript backend architect. I am providing you with 4 data models: FoodBatch, CenterProfile, DispatchRoute, and FeedbackReview. Generate the Mongoose schemas and strict Zod validation schemas for these. Then, write an Express router exposing a POST /api/batches endpoint and a PUT endpoint for uploading photos with GPS coordinates. All route responses must be wrapped in { success: boolean, data: any, error: string, code: number }. Do not generate UI code."

Module 2: Logistics & Dispatch Engine (Team Member 2)

Role: Logistics Backend Engineer & Admin Frontend Developer

Tech Stack: Node.js (Controllers/Services), Next.js (Admin Dashboard UI), Tailwind CSS.

Scope & Boundaries: You are the brain of the operation. You write the backend business logic that connects physical food to digital records via QR codes. You also build the constraint-matching algorithm that decides where food goes. On the frontend, you build the Face 4 Admin Web Dashboard for the Central NGO. You do **not** build mobile apps.

Core API Endpoints (Business Logic):

You will add these specific controller functions to Member 1's Express server.

- POST /api/logistics/match
    - **Input:** { "batchId": "uuid" }
    - **Logic:** Queries MongoDB for the batch's handlingRequirement. Queries CenterProfile for centers with matching capabilities (e.g., refrigeration: true) and sufficient maxMealCapacity. Calculates distance using basic Lat/Long math.
    - **Output:** Returns the assigned centerId and generates a DispatchRoute object.
- POST /api/logistics/link-qr
    - **Input:** { "qrStickerId": "string", "batchId": "string", "lat": number, "lng": number }

    - **Logic:** The driver scans a random physical sticker and links it to the digital batch. Updates the FoodBatch document with the qrStickerId.
    - **Output:** Returns updated FoodBatch.
- POST /api/reviews/process
    - **Input:** { "targetType": "donor", "targetId": "uuid", "rawText": "The food arrived spoiled.", "sentiment": "negative" }
    - **Logic:** Sends rawText to the LLM Gateway to extract aiParsedFlags. Saves the FeedbackReview to the database. Ratings are kept internal and never exposed to the public.

Face 4 Admin Dashboard (Frontend UI):

- **Active Routes Map:** A React component plotting active DispatchRoute locations.
- **The Priority Queue:** A data table displaying all FoodBatch items currently status: "pending" waiting for the matching algorithm.
- **Moderation Dashboard:** A view displaying FeedbackReview objects, highlighting donors with high frequencies of aiParsedFlags like "spoiled_food".

AI Prompting Instructions for Member 2:

"Act as a Logistics Backend Engineer and Next.js developer. First, write a TypeScript backend service function that takes a FoodBatch object (with hot/cold requirements) and an array of CenterProfile objects, and returns the best matching Center based on capability and proximity. Second, write a Next.js React component using Tailwind CSS that displays a data table of incoming FeedbackReview objects, keeping ratings strictly internal to the admin. Assume all API calls return a standard { success, data, error, code } wrapper."

Module 3: Face 3 Donor Frontend (Team Member 3)

Role: Event Organizer Frontend Developer

Tech Stack: Next.js (React), Tailwind CSS.

Scope & Boundaries: You are building a frictionless, mobile-first web app (PWA) for the Event Organizers (Donors). Your interface is responsible for ingesting menus, finalizing leftover inventory, and capturing the crucial initial food photos. You only interact with the backend APIs; you do **not** handle driver logic, route mapping, or consumer features.

Core Features & Workflows:

- **Pre-Event Ingestion:** A UI allowing organizers to snap a photo of their catering menu or invoice. The UI displays a loading state while calling the backend OCR endpoint, then presents a verification checklist of the extracted dishes, ingredients, and allergens.
- **Post-Event Triage:** A streamlined checklist UI pre-loaded with the verified menu. The organizer selects which items have leftovers, estimates the number of servings, and

uses the device camera to take the "Stage 1" photo.
- **Predictive Logistics Display:** If the organizer forgets to specify the transport requirements (e.g., refrigeration), you must build UI feedback showing that the LLM has automatically inferred the requirement from the food type (e.g., "Ice cream detected -> Requires deep freeze transport").

API Endpoints Consumed:

- POST /api/llm/ocr -> Sends the menu photo, receives the structured JSON array of dishes.
- POST /api/batches -> Submits the finalized leftovers array to create a new FoodBatch.
- PUT /api/batches/:batchId/photo -> Uploads the Stage 1 photo. Must include GPS Lat/Long from the device.

AI Prompting Instructions for Member 3:

"Act as a Senior Next.js Frontend Developer. I am building the 'Donor Face' of a food rescue platform. Generate a mobile-first PWA using Tailwind CSS with two main flows: 1) A menu upload UI that hits a POST /api/llm/ocr endpoint and renders the returned JSON array of dishes into a checklist. 2) A leftover triage form that allows the user to estimate servings, take a photo (using HTML5 camera APIs), capture GPS coordinates, and submit the payload to a POST /api/batches endpoint. Ensure all state is managed locally via React hooks."

Module 4: Center, Driver, & Consumer Mockup (Team Member 4)

Role: Logistics & Consumer Frontend Developer

Tech Stack: Next.js (React), Tailwind CSS, Google Maps API (Mocked).

Scope & Boundaries: You have the most diverse frontend role. You are building three isolated interfaces: the Driver's mobile view (Face 4), the Distribution Center's intake tablet/mobile view (Face 2), and the static mockup of the Consumer app (Face 1). You do **not** write backend matching algorithms or database schemas.

Core Features & Workflows:

- **The Driver Workflow (Face 4):** A mobile-optimized view displaying the driver's active route. It requires a QR scanner component (using a React QR reader library). The driver scans the Event QR, takes the "Stage 2" pickup photo, and scans one of their pre-printed, blank QR tracking stickers to physically tag the container.
- **The Center Intake (Face 2):** A web app where Center staff scan the driver's QR sticker upon arrival and take the final "Stage 3" verification photo of the arriving food. It includes a feedback form allowing staff to submit reviews on specific food batches (e.g., spoiled

deliveries or inappropriate packaging).

- **The Consumer Mockup (Face 1):** A static UI built solely to impress judges. It must show a dropdown for FDA-recognized allergens, a text box for custom dietary needs (e.g., "I avoid high sodium"), and a Google Maps UI indicating nearby centers. It includes a Thumbs Up/Down and text review screen, but you must enforce the rule that ratings remain internal and are never displayed publicly on the venue profiles.

API Endpoints Consumed:

- POST /api/logistics/link-qr -> (Driver View) Links the scanned blank sticker to the digital batch.
- PUT /api/batches/:batchId/photo -> (Driver & Center Views) Uploads Stage 2 and Stage 3 photos with GPS coordinates.
- POST /api/reviews/process -> (Center View) Submits negative feedback directly to the central NGO moderation queue.

AI Prompting Instructions for Member 4:

"Act as a Senior Next.js Frontend Developer. I need three distinct mobile-first UI components using Tailwind CSS: 1) A 'Driver View' containing a QR code scanner UI that links a physical sticker ID to a batch via POST /api/logistics/link-qr, and a camera component to upload a photo with GPS coordinates. 2) A 'Soup Kitchen Intake View' that allows staff to scan a QR code, upload an arrival photo, and submit a feedback form to POST /api/reviews/process. 3) A purely static, hardcoded 'Consumer Mockup' UI that displays a dietary allergen dropdown, a fake Google Maps view, and a Thumbs Up/Down review screen. Do not intermingle the state of these three apps."